



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Árboles
UOC:	BÁSICA
Nivel y Paralelo:	Tercero - B
Nombre:	Sivinta Lalsuisa Cristian Fernando
	
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles.

Específicos:

- Utilizar operaciones fundamentales sobre árboles N-arios y árboles binarios utilizando recursividad.
- Implementar recorridos (In-Order) y transformaciones (inversión) para manipular la estructura de árboles.
- Distinguir el comportamiento y aplicación de árboles N-arios vs Árboles Binarios de Búsqueda (BST).

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de baja complejidad utilizando árboles.



1. INTRODUCCIÓN

En esta sección se presentan los resultados obtenidos tras la implementación y ejecución de los cuatro recorridos de árboles binarios (Inorden, Preorden, Postorden y BFS) en los lenguajes C++ y Java. Los resultados se organizan por ejercicio, mostrando tanto los valores teóricos esperados como las capturas de pantalla de las ejecuciones reales. Esta comparación permite verificar la correcta implementación de los algoritmos y validar su funcionamiento con diferentes estructuras de árbol, desde el árbol base hasta el árbol modificado con nodos adicionales y el caso aplicado del sistema web.

2. MARCO TEÓRICO

Los recorridos de árboles binarios son métodos sistemáticos para visitar todos los nodos de una estructura jerárquica [1]. Los recorridos en profundidad (DFS) incluyen:

- **Preorden:** Visita primero la raíz, luego el subárbol izquierdo y finalmente el derecho. Es útil para copiar o serializar árboles [2].
- **Inorden:** Visita primero el subárbol izquierdo, luego la raíz y finalmente el derecho. En un Árbol Binario de Búsqueda (BST), produce los elementos en orden ascendente [3].
- **Postorden:** Visita primero el subárbol izquierdo, luego el derecho y finalmente la raíz. Es útil para eliminar árboles o procesar hijos antes que padres [2].

El recorrido en amplitud (BFS) visita los nodos nivel por nivel de izquierda a derecha utilizando una cola como estructura auxiliar, lo que permite mostrar la estructura jerárquica por niveles [4].

A continuación, se presentan los 5 ejercicios implementados en lenguaje C++ y Java, junto con sus respectivos códigos, árboles de prueba y salidas obtenidas.

2.1. Ejercicio 1: Conteo de nodos en árbol N-ario

Descripción: Implementar una función que recorra recursivamente un árbol N-ario y cuente el número total de nodos.

Código implementado (C++):

```
int contarNodos(NodoN* raiz) {  
    if (raiz == nullptr) return 0;  
    int total = 1;  
    for (NodoN* hijo : raiz->hijos) {  
        total += contarNodos(hijo);  
    }  
    return total;  
}
```

Código implementado (Java):

```
public static int contarNodos(NodoN raiz) {  
    if (raiz == null) return 0;  
    int total = 1;  
    for (NodoN hijo : raiz.hijos) {  
        total += contarNodos(hijo);  
    }  
    return total;  
}
```



Captura de ejecución:

```
--- Prueba Ejercicio 1 ---  
Nodos esperados: 6  
Nodos calculados: 6
```

2.2. Ejercicio 2: Inserción en Árbol Binario de Búsqueda (BST)

Descripción: Implementar inserción en BST respetando la propiedad: valores menores a la izquierda, mayores o iguales a la derecha.

Código implementado (C++):

```
Nodo* insertar(Nodo* raiz, int valor) {  
    if (raiz == nullptr) return new Nodo(valor);  
    if (valor < raiz->valor)  
        raiz->izquierdo = insertar(raiz->izquierdo, valor);  
    else  
        raiz->derecho = insertar(raiz->derecho, valor);  
    return raiz;  
}
```

Código implementado (Java):

```
public static Nodo insertar(Nodo raiz, int valor) {  
    if (raiz == null) return new Nodo(valor);  
    if (valor < raiz.valor)  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    else  
        raiz.derecho = insertar(raiz.derecho, valor);  
    return raiz;  
}
```

Captura de ejecución:

```
--- Prueba Ejercicio 2 ---  
Raiz (Esperado 10): 10  
Hijo Izquierdo de Raiz (Esperado 5): 5  
Hijo Derecho de Raiz (Esperado 15): 15  
Hijo Izquierdo del 5 (Esperado 3): 3
```

2.3. Ejercicio 3: Cálculo de profundidad máxima (altura)

Descripción: Calcular la altura del árbol (número de nodos en la ruta más larga desde raíz hasta hoja).

Código implementado (C++):



```
int calcularAltura(Nodo* raiz) {
    if (raiz == nullptr) return 0;
    int alturaIzq = calcularAltura(raiz->izquierdo);
    int alturaDer = calcularAltura(raiz->derecho);
    return 1 + max(alturaIzq, alturaDer);
}
```

Código implementado (Java):

```
public static int calcularAltura(Nodo raiz) {
    if (raiz == null) return 0;
    int alturaIzq = calcularAltura(raiz.izquierdo);
    int alturaDer = calcularAltura(raiz.derecho);
    return 1 + Math.max(alturaIzq, alturaDer);
}
```

Captura de ejecución:

2.4. Ejercicio 4: Recorrido In-Order

Descripción: Implementar recorrido in-order (izquierda → nodo → derecha) que produce elementos ordenados en un BST.

```
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
```

Código implementado (C++):

```
inOrderAux(Nodo* nodo, vector<int>& resultado) {
    if (nodo == nullptr) return;
    inOrderAux(nodo->izquierdo, resultado);
    resultado.push_back(nodo->valor);
    inOrderAux(nodo->derecho, resultado);
}
```

Código implementado (Java):

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
    if (nodo == null) return;
    inOrderAux(nodo.izquierdo, resultado);
    resultado.add(nodo.valor);
    inOrderAux(nodo.derecho, resultado);
}
```

Captura de ejecución:



```
--- Prueba Ejercicio 4 ---  
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]  
Tu resultado:      [1, 2, 3, 4, 5, 6, 7]
```

2.5. Ejercicio 5: Inversión del árbol (árbol espejo)

Descripción: Transformar el árbol intercambiando recursivamente los hijos izquierdo y derecho de cada nodo.

Código implementado (C++):

```
Nodo* invertir(Nodo* raiz) {  
    if (raiz == nullptr) return nullptr;  
    Nodo* temp = invertir(raiz->izquierdo);  
    raiz->izquierdo = invertir(raiz->derecho);  
    raiz->derecho = temp;  
    return raiz;  
}
```

Código implementado (Java):

```
public static Nodo invertir(Nodo raiz) {  
    if (raiz == null) return null;  
    Nodo temp = invertir(raiz.izquierdo);  
    raiz.izquierdo = invertir(raiz.derecho);  
    raiz.derecho = temp;  
    return raiz;  
}
```

Captura de ejecución:

```
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```

3. ANÁLISIS DE RESULTADOS

Los 5 ejercicios fueron implementados exitosamente tanto en C++ como en Java. Se desarrollaron habilidades fundamentales en el trabajo con árboles:



- Recursividad: Todos los ejercicios utilizaron recursion como tecnica principal para recorrer y transformar los árboles, demostrando que es la aproximacion mas natural para estructuras jerarquicas.
- Arboles N-arios vs Binarios:
 - Se evidencio la diferencia estructural fundamental:
 - **Arbol N-ario:** cada nodo contiene un vector de hijos
 - **Arbol binario:** cada nodo tiene maximo 2 hijos (izquierdo y derecho)
- Propiedad BST: La insercion en BST respeta la propiedad de orden, permitiendo busquedas eficientes en $O(\log n)$.
- Recorrido In-Order: Sobre un BST produce automaticamente una secuencia ordenada de menor a mayor.
 - Transformación estructural: La inversion modifica completamente la forma del arbol intercambiando recursivamente los punteros de los hijos.

3.1. Comparativa de resultados entre lenguajes

Aspecto	C++	Java
Tiempo de ejecución	Más rápido (código nativo)	Más lento (JVM)
Consumo de memoria	Menor (control manual)	Mayor (Garbage Collector)
Facilidad de implementación	Moderada (manejo de punteros)	Alta (sin punteros)
Portabilidad	Requiere recompilación	Multiplataforma
Depuración	Más compleja (punteros nulos)	Más sencilla (excepciones)

3.2. Aprendizajes clave

DFS vs BFS: Se comprendió la diferencia fundamental entre la búsqueda en profundidad (usando recursividad/pila) y la búsqueda en amplitud (usando cola). BFS es ideal para mostrar estructuras por niveles, mientras que DFS es más eficiente para búsquedas profundas.

Aplicaciones prácticas:

- Preorden: Ideal para serializar árboles y mostrar menús jerárquicos.
- Inorden: Perfecto para obtener datos ordenados en un BST.
- Postorden: Útil para eliminar árboles y procesar hijos antes que padres.
- BFS: Excelente para visualizar estructuras por niveles y encontrar caminos más cortos.

Importancia del orden: El orden en que se visitan los nodos determina la utilidad del recorrido, lo que quedó demostrado en el caso aplicado del sistema web.

4. CONCLUSIONES DEL TRABAJO

La implementación de los cuatro recorridos en ambos lenguajes permitió comprender las ventajas y desventajas de cada uno. C++ ofrece mayor control y rendimiento, mientras que Java facilita la implementación con su gestión automática de memoria. Los resultados obtenidos coinciden con los valores teóricos esperados, validando la correcta implementación de los algoritmos recursivos y el uso de la cola para BFS. La aplicación al caso del sistema web demostró la utilidad práctica de los recorridos en escenarios reales de desarrollo de software.



2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- La implementación de los recorridos y operaciones sobre árboles en C++ y Java permitió comprobar el correcto funcionamiento de los algoritmos de búsqueda, inserción y transformación de estructuras jerárquicas.
- El uso de recursividad facilitó el desarrollo de soluciones eficientes para trabajar con árboles binarios y N-arios, demostrando su importancia en el manejo de estructuras de datos.
- La comparación entre C++ y Java evidenció diferencias en rendimiento, manejo de memoria y facilidad de programación, permitiendo identificar ventajas y desventajas de cada lenguaje en la implementación de algoritmos.

2.10 Recomendaciones

- La recursividad es una herramienta poderosa para recorrer y transformar árboles.
- El BST facilita búsquedas eficientes si se mantiene la propiedad de orden.
- El recorrido in-order sobre un BST produce una secuencia ordenada.

2.11 Referencias bibliográficas

- [1] “Trees in Data Structure | What is Trees in Data Structure?” Accessed: May 12, 2026. [Online]. Available: <https://www.mygreatlearning.com/blog/understanding-trees-in-data-structures/>
- [2] “Types of Trees in Data Structure: Applications and Benefits | Intellipaat.” Accessed: May 12, 2026. [Online]. Available: https://intellipaat-com.translate.goog/blog/types-of-trees-in-data-structures/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- [3] “Estructura de datos - Árboles - Oscar Blancarte - Software Architecture.” Accessed: May 12, 2026. [Online]. Available: <https://www.oscarblancarteblog.com/2014/08/22/estructura-de-datos-arboles/>
- [4] “DSA Binary Trees.” Accessed: May 12, 2026. [Online]. Available: https://www.w3schools.com/dsa/dsa_data_binarytrees.php
- [5] “Estructuras de datos: Una guía completa con ejemplos en Python | DataCamp.” Accessed: May 12, 2026. [Online]. Available: <https://www.datacamp.com/es/tutorial/data-structures-guide-python>

2.12 Anexos

LINK DE GITHUB: https://github.com/Cristian-1402/APE3_ARBOLES.git